

Natural-Language Supervision of a 20-Robot Swarm: An LLM Orchestration Layer over Classical MAPF and Formation Control

Evgenii Shlomov Artyom Tuzov Senya Belyh Dmitry Ryabov
Dmitriy Vizitei Anastasia Malakhova

Innopolis University, Innopolis, Russia

{e.shlomov, a.tuzov, s.belykh, d.ryabov,
d.vizitei, a.malakhova@innopolis.university}

Abstract

We present `iros_llm_swarm`, a complete ROS 2 system in which a large language model supervises a 20-robot differential-drive swarm from free-form natural language. Unlike prior LLM-robotics work that emits low-level code or single-robot skill sequences, our orchestration layer emits a *typed, recursive plan tree* over five fleet-level primitives, and every leaf is discharged by a classical algorithm — Multi-Agent Path Finding (MAPF) with two interchangeable backends (PBS and LNS2), a centralized leader–follower formation manager, and a BehaviorTree.CPP mission front-end. The LLM therefore never touches velocities, trajectories, or conflict resolution; it is confined to a small, well-typed decision surface that the underlying stack can validate, reject, and repair.

The contribution of this paper is the orchestration layer and its evaluation methodology. We describe three control channels (reactive, proactive, and operator chat), a grounding stack that gives the model bounded read-only access to live fleet state through eleven semantic tools, schema-constrained decoding against the plan grammar, and—centrally—a *defence-in-depth* design in which four deterministic guards and three LLM-in-the-loop repair loops sit between a generated plan and the robots. We further contribute a reproducible evaluation design: a 44-case hermetic planning benchmark spanning twelve structural categories, and a proximity-driven task system that turns end-to-end mission success into an automatically scored quantity, so that planning accuracy and mission completion can be measured separately and attributed to individual architectural components by ablation.

[PENDING RESULTS: This draft fixes the system description and the experimental design. E1 (hermetic planning accuracy) is populated across five local models; E2–E6 remain unpopulated pending further runs.]

1 Introduction

Coordinating a fleet of mobile robots that share one workspace is a long-standing problem. The classical decomposition separates fleet-level decisions—which robot goes where, in what order—from robot-level execution—how one robot follows a

path or holds an offset. Large language models make this decomposition newly attractive [?, ?, ?, ?]: an LLM can reason over compact semantic primitives instead of producing control commands, while the classical stack retains responsibility for safety, conflict resolution, and real-time tracking.

The difficulty is that this promise is easy to state and hard to make hold. A language model asked to command twenty robots will confidently emit coordinates inside walls, address robots that are currently locked into a formation, place four robots on the same point, invent room names, and return JSON that does not parse. Any one of these silently produces a fleet-wide failure. The engineering question this paper addresses is therefore not *can an LLM plan for a swarm* but *what has to sit between the LLM and the robots so that it can*, and how much each of those pieces is actually worth.

1.1 Scope and contributions

We study a fleet of 20 differential-drive robots in a 2D environment simulated with Stage, each running its own Nav2 stack. The system spans three layers: a natural-language orchestration layer (this paper’s focus), a classical coordination substrate (MAPF, formations, behavior trees), and per-robot navigation. Our contributions:

- (1) **A typed plan language for fleet control.** Five leaf primitives (`mapf`, `formation`, `disband`, `idle`) and two containers (`sequence`, `parallel`) form a recursive tree that is expressive enough for multi-group, multi-phase missions yet small enough to be validated exhaustively and to be expressed as a JSON schema for constrained decoding (Sec. ??).
- (2) **Three orchestration channels** over one substrate: a reactive channel invoked by behavior-tree nodes on WARN/ERROR, a proactive channel that observes fleet state and intervenes on its own initiative, and an operator chat channel that streams a reply and an executable plan (Sec. ??).
- (3) **A grounding stack** that replaces coordinate hallucination with lookup: a live pose cache, a room-occupancy-aware group placement solver, and eleven read-only semantic tools brokered behind an allowlist, with an optional MCP provider for raw ROS introspection (Sec. ??).
- (4) **Defence in depth between plan and robots:** schema-

constrained decoding, permissive type coercion, an active-formation guard, an occupancy-aware goal rewriter, and automatic formation staging—four of which are deterministic and none of which require another model call (Sec. ??).

- (5) **A bounded closed loop:** deterministic post-execution verification feeding three nested LLM repair loops (remediation, verification repair, mission supervision), each with an explicit step, time, and no-progress budget (Sec. ??).
- (6) **A 20-robot execution substrate** that makes the above runnable and measurable: a Stage 2D environment reached after a documented pivot from Gazebo Harmonic, five scenarios with symbolic map descriptions and a dynamic obstacle system, per-robot Nav2 stacks generated from a single parameter template with staggered bring-up, and a static-localization strategy that replaces AMCL at this fleet density (Sec. ??).
- (7) **A reproducible evaluation design** (Sec. ??) that separates planning competence from execution competence, scores end-to-end missions automatically via a proximity-driven task system, and attributes outcomes to individual components through configuration-level ablation.

The system was previously documented as two partial technical reports written against a three-level split of the work: one covering the classical coordination substrate [?], the other the simulation environment, per-robot navigation, and an early design of the orchestration layer [?]. Each necessarily described its layer against *assumed* interfaces at the boundary. This paper is the unified account of the system as built: it supersedes the evaluation sections of both reports, corrects the orchestration-layer description to match the shipped implementation, and—because planning quality and mission success can only be compared on the whole stack—evaluates the three layers together rather than separately.

2 Related Work

Multi-Agent Path Finding. MAPF has a rich algorithmic literature [?]. Conflict-Based Search [?] is the canonical optimal solver. Priority-Based Search (PBS) [?] relaxes optimality for scalability: a strict priority ordering replaces time-indexed constraints, and each agent plans deterministically against higher-priority paths. Large Neighborhood Search for MAPF (LNS2) [?] is anytime and suboptimal, starting from a possibly colliding plan and iteratively destroying and repairing agent subsets with soft-constraint A*. We adopt both behind one action interface, which lets us treat the backend as an experimental factor rather than a design commitment.

Formation control. Formation control is typically posed as leader–follower, virtual structure, or behavior-based [?, ?]. We use leader–follower because it composes cleanly with a per-robot motion controller and matches the kind of semantic offset (“a wedge”, “abreast”) a language model naturally produces.

Navigation and behavior trees. Nav2 [?] is the de facto ROS 2 navigation stack, with layered costmaps [?] the standard obstacle representation. Behavior trees [?] via BehaviorTree.CPP [?] provide a modular control architecture that is particularly suitable when an external reasoning component supplies tree parameters.

LLMs for robotics. SayCan [?] grounds language in affordances; Code as Policies [?] and ProgPrompt [?] emit executable program text; Inner Monologue [?] closes the loop with textual feedback. Our design differs in two respects that matter at fleet scale. First, the model emits a *typed tree over fleet primitives*, not code: the output space is small enough to constrain by schema and validate exhaustively, which removes the arbitrary-code-execution surface and the malformed-output failure mode. Second, feedback is not free text but a *deterministic verification record* computed from formation status and pose state (Sec. ??), so the repair loop is driven by measured ground truth rather than by the model’s own narration.

ROS 2 and DDS at scale. DDS discovery cost dominates at scale [?, ?]. We use CycloneDDS with a project-specific profile; without it the 20-robot system does not reliably complete discovery within the launch budget.

3 The Classical Substrate

This section summarizes the layer the orchestrator commands; see [?] for full detail.

Shared fleet action. Both MAPF backends own the same long-lived `/swarm/set_goals` action, which encapsulates the entire *validating* $\rightarrow$ *planning* $\rightarrow$ *executing* $\rightarrow$ *replanning* $\rightarrow$ *succeeded* lifecycle behind one goal token. Schedule monitoring and replanning are internal; clients see one atomic operation. Cancellation stops every robot. The action result carries a full metrics payload—planning time, node expansions, per-agent path lengths, replan count, total execution time—which we exploit directly for benchmarking (Sec. ??).

PBS backend. An 8-connected A* on a grid downsampled to 0.2 m, with a backward-Dijkstra heuristic per unique goal, two-zone gradient inflation (hard footprint plus a soft penalty zone), and capsule-vs-capsule conflict detection that subsumes the usual vertex/edge split. The high-level search follows the priority-based formulation of [?].

LNS2 backend. An initial soft-constraint A* solution refined by an adaptive large-neighborhood search with four destroy operators (collision-based, random-walk, random, bottleneck) whose weights adapt online. It is anytime, tolerant of transiently infeasible starts, and warm-starts on replan.

Per-robot followers and formations. Each robot runs a dual-mode follower: in AUTONOMOUS mode it dispatches its assigned plan to Nav2 `follow_path`; in FORMATION mode it disengages Nav2 and runs a PD controller tracking $p_{\text{ref}} = p_{\text{leader}} + R(\theta_{\text{leader}})o$ for a body-frame offset o . Mode switching is driven by a single latched message, without node restarts. A centralized manager owns formation definitions and republishes the full registry on a latched topic; a monitor computes per-follower error $\varepsilon_i = \|p_i - (p_{\text{leader}} + R(\theta_{\text{leader}})o_i)\|$ and classifies each formation as INACTIVE / FORMING / STABLE / DEGRADED / BROKEN at 10 Hz. These ε_i are published, which makes formation tracking accuracy directly measurable (Sec. ??).

Behavior-tree front-end. BehaviorTree.CPP v3 nodes wrap every primitive: `MapfPlan`, `SetFormation`, `DisableFormation`, and a `CheckMode` guard. Mission completion is observed on the `/bt/state` blackboard mirror, whose `active_action` field cycles `none` $\rightarrow$ `node` $\rightarrow$ `none`; the orchestrator keys leaf completion on this transition rather than on the command acknowledgement, which is only an ack.

Simulation environment. The fleet runs in Stage 2D [?] via `stage_ros2`, with a unified TF tree and enforced namespace prefixes. An earlier Gazebo Harmonic [?] configuration was implemented end-to-end and then retired: per-robot physics, collision broadphase, and sensor rendering pushed the real-time factor below usable thresholds at 20 robots before any navigation stack was attached [?]. The Gazebo configuration is retained in tree for smaller fleets and future hardware-realistic preview.

Five scenarios share one robot model and spawn protocol: `cave` (75×75 m, narrow single-file passages), `large_cave` (150×150 m, same topology at double scale), `warehouse_2` and `warehouse_4` (30×30 m, two- and four-group symmetric layouts), and `amongus` (66.6×37.75 m), which maps the spaceship layout of *Among Us: The Skeld* into 14 named rooms joined by narrow corridor choke-points. We use `amongus` for all orchestration experiments because its rich navigation exercises symbolic reference and its choke-points stress conflict resolution.

Each scenario ships a machine-readable map description consumed by the orchestrator: named locations with coordinates, aliases in English and Russian, colour-coded robot groups with home and spawn positions, formation-capable zones with free-space radii, and map-specific planning heuristics (bottleneck locations, known-unreachable goal patterns, abort-versus-replan guidance). A dynamic obstacle manager additionally supports runtime-toggable doors and placeable obstacles; `amongus` declares six corridor doors matching the map geometry.

Scale engineering. Three decisions are load-bearing at 20 robots and are reported because they constrain the experiments. CycloneDDS with a custom profile is mandatory above

roughly ten participants. AMCL is disabled—each robot’s scan contains every other robot’s body, and particle weights collapse—so localization is a static per-robot `map` $\rightarrow$ `odom` identity transform over simulator odometry. Bring-up is staggered (simulator 0 s, Nav2 1 s, planner 10 s, followers 12 s, plus 0.3 s per Nav2 instance).

4 Orchestration Layer

4.1 Three channels

The orchestrator exposes three independent paths from language to fleet action, which share prompt infrastructure and the plan validator but differ in trigger, latency budget, and authority.

Channel 1 (reactive). A `/llm/decision` action server. Behavior-tree nodes call it when a primitive reports WARN or ERROR, supplying the event plus a tail of the accumulated log. The model returns exactly one of `wait` / `abort` / `replan`. The prompt biases hard toward `wait` and enumerates planner-internal warnings that indicate *self-recovery in progress* (“replan did not reach collision-free; keeping old paths”, “warm seed rejected”) as explicit `wait` cases—because the expensive error here is a spurious `replan` that cancels a plan the substrate was about to fix on its own. Always active.

Channel 2 (proactive). A passive observer subscribing to `/bt/state`. It decides on its own initiative whether to intervene and publishes an `LlmCommand` that a behavior-tree receiver applies to the blackboard. It is cooldown-gated and checks an `llm_thinking` flag to avoid colliding with channel 1. Off by default.

Channel 3 (operator chat). The main path, and the focus of this paper. An operator types free-form text; the server streams a natural-language reply token-by-token, parses an executable plan from the same response, and dispatches it. Everything in Sections ??–?? describes channel 3.

4.2 Plan language

Channel 3 returns a single JSON object `{reasoning?, reply, plan}`, where `plan` is a recursive tree over the node types in Table ??.

Three design decisions in this grammar are worth stating explicitly.

Parallelism is a claim about intent, not about threads. A `parallel` node is flattened before execution: multiple `mapf` leaves *merge into one* `mapf` goal, because the MAPF planner already resolves inter-robot conflict internally and issuing two concurrent fleet goals would defeat exactly the coordination we want. Mixed `mapf` and `formation` children degrade to sequential execution, since a formation requires its own follower mode. An `idle` in any branch short-circuits the whole node—stop takes priority. This means the operator’s “do A and B at the same time” is honoured as *one jointly planned motion*, which is stronger than concurrent dispatch, not weaker.

idle doubles as the escalation channel. Rather than adding node types for “I need clarification” and “here is an answer, no

Table 1: Plan tree node types.

Node	Kind	Payload / semantics
mapf	leaf	robot_ids, goals, spread; one goal per robot, or one centre that the server grids
formation	leaf	formation_id, leader_ns, follower_ns, offsets_x/y
disband	leaf	formation_id
idle	leaf	reason; also the carrier for escalation (see below)
sequence	container	children in order; abort on first failure
parallel	container	see flattening rules below

action needed”, the model emits `idle` with a prefixed reason: `needs_help:`, `clarify:`, or `reply_only:`. This keeps the grammar at six node types—small enough to constrain by schema—while still distinguishing refusal-to-act from action. It also makes escalation a first-class, testable behaviour: a benchmark case can assert that a question with a missing robot pose produces `needs_help:` and not a confident plan over stale data.

The schema is enforced twice. `plan_schema.py` expresses the grammar as a JSON schema with a recursive `$defs/node` reference, passed to the backend for constrained decoding (Ollama format, OpenAI `response_format`). But JSON Schema cannot express the cross-field constraint `|goals| = |robot_ids|`, so a hand-written validator re-checks every node after parsing. Constrained decoding removes the *malformed* failure mode; the validator removes the *incoherent* one.

4.3 Grounding

A model asked for “cyan to the cafeteria” must resolve a colour group to robot ids, a room name to coordinates, and coordinates to free floor space. We push all three out of the model’s head and into lookup.

Static context. The prompt carries a per-map description: named locations with coordinates, colour groups with member ids and home positions, formation-capable zones with radii, and door geometry. Maps are declared in `common_scenarios.yaml`, so adding an environment does not touch the orchestrator.

Live context. A pose cache subscribes to `/robot_*/odom` and supplies fresh positions with an explicit staleness bound (2 s). This is what lets the model compute formation goals as `leaderxy + offsets` rather than inventing absolute coordinates—and, equally important, lets it *notice a missing robot* and escalate instead of planning over a gap. Formation status, behavior-tree state, recent events, and open task state are folded into the same snapshot under a character budget.

Semantic tools. Eleven read-only tools sit above raw introspection: system state, group state, location resolution, route context with bottleneck and door penalties, goal and formation feasibility checks, occupancy-aware free group-goal search in-

Table 2: Layers between a generated plan and the fleet. D = deterministic, L = requires an LLM call.

Layer		Failure mode addressed
Constrained decoding	D	unparseable / off-grammar JSON
Schema validator	D	structurally valid but incoherent nodes
Type coercion	D	"robot_10" where an int is required
Active-formation guard	D	direct MAPF control of a robot locked in a live formation
Occupancy rewrite	D	room-centre goals that land in walls or clump
Formation prestaging	D	bare <code>formation</code> leaf with followers out of position
Remediation loop	L	execution failed; replan with failure briefing
Verification repair	L	execution reported success but the outcome does not hold
Mission supervision	L	multi-step mission needs continuation

side a room, post-execution verification, the allowed action schema, and recovery options. They are dispatched by a broker that enforces an allowlist, a per-round tool cap, a timeout, and a result-size cap. An optional MCP provider (`ros-mcp` over `stdio`) adds raw ROS introspection under a separate allowlist restricted to read verbs. The model never sees MCP tools directly—only a bounded snapshot or a brokered semantic call—so the write surface is structurally absent rather than merely discouraged.

Notably, tool calling and schema-constrained decoding are *mutually exclusive* in current backends: constrained decoding suppresses tool-call emission. The system therefore has two distinct planning modes, which we treat as a primary experimental factor (Sec. ??).

4.4 Defence in depth

Table ?? lists what stands between a generated plan and the robots. The design principle is that *a deterministic guard is always preferred to another model call*: guards are cheap, reproducible, and cannot themselves hallucinate.

Two of these deserve elaboration because they encode domain knowledge the model does not reliably have.

Type coercion is deliberately permissive. Small local models routinely emit "robot_10" or 10.0 where the grammar wants 10. A strict parser would reject an otherwise perfect plan over a namespace string. The coercer accepts 10, "10", "robot_10", "robot10", and integral floats, while explicitly rejecting booleans (which are `int` subclasses in Python and would otherwise silently become robot 0 or 1). Strictness is spent where it buys safety, not where it only punishes formatting.

The *active-formation guard* exists because “send the magenta line to storage” is ambiguous in a way that matters. A formation is commanded by moving its *leader*; commanding all four members individually tears it apart. The guard reads live formation status, and for any formation in FORMING, STABLE, or DEGRADED it strips follower ids out of MAPF leaves, leaving the leader. It returns a structured failure record shaped exactly like a PlanExecutor failure, so the existing repair loop consumes it with no special handling.

4.5 Closed loop

Execution success is not the same as mission success, and this distinction drives the whole loop design. A plan can execute cleanly—every action server returns success—while the requested outcome does not hold: robots arrive but the formation never stabilizes, or they arrive at a room’s centre and pile up. The system therefore verifies deterministically after execution, reading formation status and pose state to decide whether what was asked for is actually true, and emits an honest *partial* result when state is missing rather than guessing.

Three nested loops consume that verdict, each explicitly budgeted:

- **Remediation** (default 2 attempts) fires when execution *fails*. It re-prompts with the failed leaf, the error, the history of prior attempts, and a *targeted refresh* of runtime context—not the full snapshot, to keep the retry prompt inside the context budget.
- **Verification repair** (default 2 attempts) fires when execution *succeeds* but verification says the outcome does not hold, and only when the verifier itself marks the discrepancy repairable.
- **Mission supervision** (default 6 steps, 180s) wraps both in a bounded execute–verify–continue loop for multi-phase missions, with a no-progress limit of 2 and a fingerprint set of already-failed plans so the model cannot loop on a plan that has already been tried.

Every loop terminates in a help escalation to the operator rather than in silent failure or unbounded retry. We regard these budgets as tuning parameters, not constants, and measure their effect directly (Sec. ??).

4.6 Task system as mission ground truth

A separate node maintains a set of spatial tasks and polls TF at 5 Hz. A *point* task transitions PENDING → DONE when any robot enters its radius. A *carry* task transitions PENDING → CARRYING on pickup and → DONE when the carrying robot reaches the dropoff. Tasks are declared per scenario in YAML and their state is published.

This component was built for mission planning, but it doubles as the measurement instrument that makes end-to-end evaluation tractable: it converts “did the swarm accomplish what the operator asked” into an automatically scored, timestamped, operator-independent quantity. Every end-to-end experiment below is scored from this signal.

Table 3: Metrics and their existing sources.

Quantity	Source (already implemented)
Plan validity, category accuracy, latency	44-case benchmark harness
MAPF planning time, expansions, path lengths, replans	SetGoals action <i>result</i> payload
Arrival / deviation / stall counts	SetGoals action <i>feedback</i>
Formation error ε_i , state machine	FormationStatus @ 10 Hz
Mission completion, time-to-complete	TaskState from the task manager
Repair / remediation / supervision counts	channel-3 event stream

5 Experimental Design

This section is the methodological core of the paper. We state it in full before reporting numbers so that the measurement protocol is fixed independently of its outcome.

5.1 Research questions

- RQ1** How accurately does a locally-hosted open-weight LLM translate free-form fleet commands into valid plans, and how does accuracy vary across linguistic and structural categories?
- RQ2** What does each grounding and constraint mechanism contribute? Specifically: schema-constrained decoding vs. tool calling, live pose context, and context budget.
- RQ3** Does planning accuracy predict mission success, or does the gap between them—the part closed by guards and repair loops—dominate?
- RQ4** What does each closed-loop layer contribute to end-to-end mission completion, and at what cost in time and model calls?
- RQ5** How do the two MAPF backends compare under identical fleet-level requests, and does backend choice interact with orchestration quality?

RQ3 is the question we consider most important and the one the literature most often skips: a benchmark measured on plan text alone can look excellent while the fleet still fails, and the converse—weak plans rescued by strong guards—is equally possible. Measuring both on the same system is the point.

5.2 Principle: reuse existing instrumentation

A deliberate constraint on this design is that it introduces almost no new measurement code. Every metric below is read from a signal the running system already produces (Table ??). This matters for validity: an evaluation harness that reimplements the notion of success is measuring the harness, and instrumentation added for a paper tends not to survive the paper.

5.3 E1: Planning accuracy (hermetic, offline)

The 44-case benchmark calls the model directly—no simulator, no ROS—with *fixed synthetic runtime contexts* (fleet at spawn; a stable magenta line; a stable cyan wedge; robot 12’s pose missing; robot 4 at the cafeteria). Each case pairs a prompt with a programmatic validator asserting structural properties of the returned tree: node type, exact robot-id sets, goal proximity to a named location within a tolerance, goal distinctness, formation leader identity, staging-before-formation ordering, and escalation-reason prefixes.

Cases span twelve categories chosen to separate distinct competences: basic single-group `mapf`; explicit per-robot coordinates; goal spreading; `parallel` over multiple groups; multi-phase `sequence`; formation creation (which requires *staging followers first and excluding the leader*); formation movement (which requires commanding *only* the leader); `formation_complex` (creation followed by a leader-only move); `disband`; `idle`; escalation under missing or ambiguous state; and a set of edge cases (repeated/ambiguous commands, partial formations, adjacency requests). All prompts here are English. An earlier revision of this benchmark additionally scored reply-language fidelity on a parallel set of Russian-language prompts; we dropped that set because it tests a UI-level concern (does the chat reply mirror the operator’s language) orthogonal to planning accuracy, and folding it into a single pass rate obscured both signals.

Protocol. Each case is run $N=5$ times per model at temperature 0.1. We report `pass@1` (mean over repeats) and `pass@5`, per-category breakdown, end-to-end latency, and the rate of each distinct failure mode (unparseable, schema-invalid, wrong ids, wrong coordinates, wrong structure). All runs reported here use `-llm-max-tokens 2048` (raised from an initial 768, which was tight enough to make truncated JSON a confound for more verbose models independent of their planning ability) and `-llm-num-ctx 32768`.

Models. The deployment target is a single workstation with 16 GB VRAM, which fixes the candidate set. The prompt is large—we measured $\approx 9,700$ tokens for the system prompt plus seven few-shot exchanges, before runtime context—so KV-cache cost is a real constraint, not a rounding error. We sweep five locally-hosted instruction models spanning that budget: a small model (**qwen3.5:4b**, 3.4 GB); a mid-size model from a different family (**gemma3:12b**, 8.1 GB); the intended 4-bit 14B operating point (**qwen2.5:14b**, 9.0 GB); an 8-bit ≈ 9 B model (**qwen3.5-9b**, 10 GB); and the largest model that still fits under the VRAM ceiling (**mistral-small3.2:24b**, 15 GB, 4-bit, partial CPU offload expected). Whether the last of these remains practical under offload is itself part of what the latency column reports.

Threat. E1 measures planning in isolation against synthetic context. It deliberately cannot detect a plan that is structurally perfect and physically unexecutable. That is E3’s job, and the comparison between them answers RQ3.

5.4 E2: Grounding and constraint ablation

Same harness, same cases, toggling one mechanism at a time: schema-constrained decoding on/off; live pose context on/off; goal auto-spread on/off; and context budget swept across its range. All of these are existing runtime parameters, so the ablation requires no code change.

We add one factor that is not currently a parameter: *symbolic versus raw grounding*. During development we observed that prompting with raw coordinates produced a higher rate of out-of-bounds and wall-interior goals than symbolic room references, and that supplying named locations, formation zones, and map-specific heuristics substantially reduced goal-validation failures downstream [?]. That observation was uncontrolled and retrospective. Because the map description is a separable prompt block, it can be ablated cleanly—full description, locations without heuristics, and coordinates only—turning an anecdote into a measured effect size. Given that the map description is a large share of a $\approx 9,700$ -token prompt, this also quantifies what that context is buying.

One asymmetry must be stated plainly. The offline harness invokes the model directly and therefore exercises only the plain and schema-constrained paths; the *tool-calling* path lives in the chat server. Comparing constrained decoding against tool calling—the mutual exclusivity noted in Sec. ??, and arguably the single most interesting ablation here—therefore requires either extending the harness to route through the chat server, or measuring that factor only in E3. We take the first option, as the second confounds planning mode with execution noise.

5.5 E3: End-to-end missions

The primary experiment. A 20-robot stack runs in the 14-location environment with nine declared tasks (six point, three carry). An operator command is issued through channel 3; the system plans, executes, verifies, and repairs; the task manager scores completion independently.

Factors. Remediation on/off; verification repair on/off; mission supervision on/off; planning mode (constrained vs. tool-calling); model size.

Metrics. Task completion rate; time to completion; number of LLM calls per mission (the cost of each safety net); how often each guard fired; how often each repair loop fired and whether it succeeded; escalation rate; and total fleet distance travelled.

Protocol. N repeats per configuration with a fixed initial fleet state, resetting task state between runs via the existing reset service. Because a 20-robot stack is stochastic—DDS timing, Nav2 controller noise—single runs are not informative, and we report distributions, not point estimates. This directly addresses our earlier Level-2 report’s acknowledged limitation that its integration scenario was run once.

No-LLM baseline. The workspace includes a scripted behavior-tree runner that drives the identical six-step mission—fleet convergence, two formation activations, a full diagonal cross-swap, return home, idle—against the same live

20-robot stack with no LLM in the loop [?]. This is the natural ceiling for E3: it isolates what the substrate achieves when plans are known-correct by construction. The difference between scripted completion and LLM-driven completion is precisely the orchestration cost, and reporting E3 without it would leave the reader unable to tell a weak orchestrator from a hard environment.

Guard attribution. Because guards are deterministic, their contribution is measurable without disabling them: we log every firing and, for each, whether the plan would have been physically invalid without it. Disabling a safety guard on a live 20-robot fleet to measure it is neither necessary nor wise.

5.6 E3b: Channel-1 decision accuracy

Channels 1 and 2 are the least evaluated part of the system in either report, and unlike channel 3 they already write structured per-call JSONL logs (prompt, raw output, parsed verdict). This makes a replay evaluation nearly free: accumulate incident records across the E3 and E4 campaigns, have two annotators independently label the correct verdict (wait/abort/replan) from the event and log tail, and score the model against that label set with inter-annotator agreement reported.

The interesting quantity here is not overall accuracy but the *asymmetric* error rate. The prompt deliberately biases toward wait, because a spurious replan cancels a plan the substrate was about to repair on its own, whereas a spurious wait merely costs one supervision cycle. We therefore report the confusion matrix rather than a scalar, and separately the false-replan rate on the planner-internal warnings the prompt enumerates as self-recovery cases.

5.7 E3c: Recovery under environmental change

The repair loops of Sec. ?? exist for the case where the world changes after a plan is made, yet every experiment above holds the environment fixed. The dynamic obstacle manager closes this gap directly: closing one of the six *amongus* corridor doors mid-mission invalidates a committed route and forces the stack to notice, replan, and—if the goal becomes unreachable—escalate.

We script door closures at a fixed mission phase and measure: whether the MAPF layer replans without orchestrator involvement (the desired outcome, no LLM call); whether the closure instead surfaces as a channel-1 WARN and which verdict follows; time from closure to recovered motion; and mission completion compared to the undisturbed run. A door closure that makes a task *unreachable* is the sharpest available test of escalation: the correct behaviour is `needs_help`, and a confident replan onto an impossible goal is a clear failure.

Caveat. This experiment requires integration work the others do not. The obstacle manager and the zone map server both publish the map with transient-local durability and cannot currently coexist; the obstacle manager is not wired into the bring-up launches. Composing it means repointing the

static map server to the raw map topic first. We flag this as a prerequisite rather than assuming it away.

5.8 E4: MAPF backend comparison

PBS vs. LNS2 under identical fleet requests, using the existing goal-dispatch test client: convergence to a single point, random goals within a radius, and group-home dispersal, across 5/10/15/20 robots and three environments of increasing clutter. All metrics come from the action result payload. This closes our earlier Level-2 report’s “no statistics over repeated runs” gap and tests the interaction half of RQ5.

5.9 E5: Formation tracking accuracy

Recording the formation status topic while a leader traverses a fixed route yields $\varepsilon_i(t)$ directly. We characterize steady-state and peak error by shape (line, wedge, triangle, abreast) and leader speed, plus time-to-STABLE after activation and DEGRADED dwell fraction. This is a bag recording and an offline parse; our earlier Level-2 report listed it as an open gap purely because nobody had recorded it.

5.10 E6: Costmap layer ablation (secondary)

Comparing the custom self-clearing costmap layer against the stock obstacle layer under 20-robot scan density, measuring recovery-behaviour invocations and mission success. Included for completeness; lowest priority.

5.11 Instrumentation to be added

Three small additions are required, and we note them explicitly because they are the only new code the evaluation needs:

- (i) A per-mission JSONL logger on channel 3. Channels 1 and 2 already write structured logs through a shared writer; channel 3 does not, and E3 cannot be scored without it.
- (ii) Repeat and machine-readable-output flags on the benchmark harness, which currently reports only to the terminal and an exit code.
- (iii) A bag-based offline scorer for E3–E5, in preference to a live scoring node, so that runs can be re-analysed without re-execution.

5.12 Threats to validity

Simulation only. All results are in Stage 2D with simulator odometry as ground truth and AMCL disabled. Localization error, wheel slip, and real sensor noise are absent. Conclusions about orchestration transfer more readily than conclusions about tracking accuracy.

The simulator pivot is qualitative. The move from Gazebo to Stage was driven by observed infeasibility at 20 robots, not by a controlled throughput sweep. We report real-time factor as a function of fleet size and navigation load for the Stage configuration only, and make no quantitative claim about the

Table 4: E1 overall pass rates and latency (44 cases, $N=5$, 220 calls/model).

Model	pass@1	pass@5	med. (s)	p90 (s)	t/o
qwen3.5:4b	68.2%	88.6%	4.8	8.6	0
gemma3:12b	68.2%	70.5%	3.7	7.4	0
qwen2.5:14b	72.7%	79.5%	9.3	20.1	0
qwen3.5-9b	77.3%	90.9%	8.2	21.0	0
mistral-sm3.2:24b	75.0%	77.3%	29.9	52.1	0

crossover point. The same applies to bring-up reliability: staggered launch is observed to work, but failure rate as a function of stagger interval and DDS profile has not been measured over repeated cold boots.

Single host. Twenty robots on one workstation is a DDS-bound ceiling; timing metrics are contaminated by host load and should be read as relative, not absolute.

Benchmark construction. The 44 cases and the prompt were developed together by the same authors, which risks the benchmark encoding the prompt’s idiosyncrasies. The category breakdown and the E1/E3 comparison partially expose this, but we do not claim the benchmark is unbiased with respect to our own system. An independent case set would be a genuine improvement.

Model non-stationarity. Hosted models change under fixed names. We pin local weights and report quantization for every reported run.

6 Results

6.1 E1: Planning accuracy

Table ?? covers all five models swept (Sec. ??): **qwen3.5:4b** (small), **gemma3:12b** (mid-size, different family), **qwen2.5:14b** (4-bit, the intended operating point), **qwen3.5-9b** (8-bit), and **mistral-small3.2:24b** (4-bit, largest that fits). All runs use $N=5$ repeats per case (44 cases $\times$ 5 = 220 calls per model), temperature 0.1, `-llm-max-tokens 2048`, `-llm-num-ctx 32768`.

Raising the token budget from 768 to 2048 (Sec. ??) removed timeouts entirely across all five models. The best model on both metrics, qwen3.5-9b, is not the largest: parameter count does not predict accuracy here. Two further patterns stand out. First, the *smallest* model, qwen3.5:4b, is close behind on pass@1 and second-best on pass@5 (88.6%)—the gap between its pass@1 and pass@5 is the largest in the table, meaning most of its failures do not repeat under resampling and are closer to sampling noise than to a structural gap. Second, gemma3:12b and mistral-small3.2:24b show almost no such recovery (68.2% $\rightarrow$ 70.5% and 75.0% $\rightarrow$ 77.3%): their failures are largely the *same* failure on every retry, i.e. systematic rather than stochastic. Latency also does not track accuracy: mistral-small3.2:24b’s weights (15 GB, 4-bit) leave under 1 GB of headroom on the 16 GB card, forcing partial CPU offload and a median latency $6\times$ qwen3.5-9b’s—without a corresponding accuracy gain.

Table 5: E1 failure-mode counts (failed calls only, out of 220 each).

Failure mode	qwen3.5 4b	gemma3 12b	qwen2.5 14b	qwen3.5 9b	mistral sm3.2 24b
Unparseable JSON	16	1	0	6	11
Wrong structure	17	25	28	11	0
Wrong robot-id set	0	0	0	0	1
Wrong goals	11	0	10	8	5
Wrong formation fields	22	24	9	2	25
Wrong escalation reason	10	10	5	10	10
Other validator	0	8	8	1	5
Total failed / 220	76	68	60	38	57

The five models fail in qualitatively different ways (Table ??). *Wrong formation fields*—almost entirely a single validator complaint, `follower_ns(N) != offsets_x(M)` or `offsets_y(M)`, i.e. the model emits an offsets array shorter than its own follower list—is the single largest failure mode for three of five models, and for mistral-small3.2:24b it is byte-identical across 25 of its 57 failures. That repetition, combined with the model’s near-zero pass@1 $\rightarrow$ pass@5 recovery, is the signature of a deterministic generation bug in how this model fills the formation-node schema, not of noisy sampling. *Wrong structure* (choosing the wrong top-level node type, e.g. `mapf` instead of `sequence`) dominates for qwen2.5:14b (28/60 failures) and gemma3:12b (25/68); qwen3.5:4b has no single dominant mode, consistent with its noise-like failure profile above. *Wrong escalation reason* is essentially constant across four of the five models (≈ 10 failures each, qwen2.5:14b is the outlier at 5), suggesting that judging when to ask for help rather than act is uniformly hard and does not improve much with scale or family within this range—an argument for a deterministic escalation check outside the model rather than prompting harder.

By category, all five models are reliable (100% pass@1) on `idle` and `parallel`, and near-reliable on `mapf_basic` and `sequence`. `formation_create` shows the widest spread across models (100% for qwen3.5:4b down to 20% for gemma3:12b), consistent with the formation-schema failure mode above being model-specific rather than a shared difficulty. One category is weak for *every* model regardless of size or family: `mapf_spread` sits at exactly 50% pass@1 for all five—worth flagging as a possible benchmark-design artifact rather than a genuine model weakness, since the category has only two cases and a single flipped call moves the rate by 50 points; we do not read anything into it beyond that caveat. `escalate` ranges 25–75% pass@1 with no relationship to model size (qwen2.5:14b, the largest of the mid-size models, is the *worst* on this category), reinforcing the failure-mode-table observation that escalation judgment does not scale predictably.

Reading with appropriate caution. $N=5$ per case, and as few as 2 cases in some categories, is enough to separate the reliable categories from the weak ones but not enough to bound per-category confidence intervals tightly; the category-level percentages above should be read as pointing at where the failures cluster, not as precise estimates. We report them because

E2–E6 (Secs. ??–??) are designed to explain *why* those specific categories fail, not merely to confirm that they do.

[PENDING RESULTS: E2 — ablation: effect of constrained decoding, tool calling, pose context, context budget.]

[PENDING RESULTS: E3 — end-to-end mission completion by configuration, with LLM-call cost and guard/repair firing counts.]

[PENDING RESULTS: E3b — channel-1 decision confusion matrix and false-replan rate.]

[PENDING RESULTS: E3c — recovery and escalation under mid-mission door closure.]

[PENDING RESULTS: E4 — PBS vs. LNS2 across fleet size and clutter.]

[PENDING RESULTS: E5 — formation tracking error by shape and leader speed.]

7 Conclusion

[PENDING RESULTS: Full claim pending E2–E6. The intended claim remains: a small typed plan grammar plus deterministic guards lets a modestly-sized locally-hosted model supervise a 20-robot fleet, and the gap between plan-level accuracy and mission-level success is where the engineering value sits—E3 is the experiment that can actually support that sentence.]

What E1 alone supports is narrower but not nothing. First, bare planning accuracy without any of the system’s guards or repair loops sits at 68–77% pass@1 across five locally-hosted 3.4–24B models under a realistic $\approx 9,700$ -token prompt—a fleet-supervision system built on a single unguarded plan call would fail roughly one command in three to one in four *regardless of which model is deployed*, which is the quantitative case for the defence-in-depth design of Sec. ?? rather than an assumption behind it. Second, the five models’ failures are not interchangeable, and not even uniformly noisy: the smallest model’s errors look like sampling noise (pass@5 recovers most of the gap to a near-perfect pass@1), while two mid/large models repeat the *identical* validator complaint—a malformed `offsets_x/offsets_y` array in formation nodes—on retry after retry. That argues for model-specific schema-repair logic rather than a single fixed remediation budget applied uniformly. Third, escalation judgment (recognizing missing or ambiguous state and asking for help instead of acting) is weak and essentially flat across all five models independent of size or family, which is the strongest single piece of evidence in this data for keeping that decision outside the model rather than trying to prompt it better. Fourth, the categories that are hardest across the board—formation staging order, escalation judgment—are exactly the categories the plan grammar (Sec. ??) and the guard layer were built to catch downstream, which is a consistency check on the architecture rather than a result about it. Whether the guards actually close that gap in a running fleet, and whether it holds under tool calling instead of constrained decoding, is what E2 and E3 are for.

Code availability

<https://github.com/innopolis-robotics-society/>

llm_swarm

References

- [1] E. Shlomov, A. Malakhova, S. Belyh, M. Shariev, “Hierarchical Multi-Agent Control by LLM: Level-2 Swarm Coordination via MAPF, Formation Control, and Behavior Trees,” technical report, 2026.
- [2] R. Stern et al., “Multi-agent pathfinding: Definitions, variants, and benchmarks,” SoCS, 2019.
- [3] G. Sharon et al., “Conflict-based search for optimal multi-agent pathfinding,” *Artificial Intelligence*, 2015.
- [4] H. Ma et al., “Searching with consistent prioritization for multi-agent path finding,” AAAI, 2019.
- [5] J. Li et al., “MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search,” AAAI, 2022.
- [6] C. Jiang, Z. Chen, and Y. Guo, “Multi-robot formation control: a comparison between model-based and learning-based methods,” *J. Control Decis.*, vol. 7, no. 1, pp. 90–108, 2020.
- [7] E. Debie, K. Kasmarik, and M. Garratt, “Swarm robotics: A survey from a multi-tasking perspective,” *ACM Comput. Surv.*, vol. 56, no. 2, 2023.
- [8] H. Wang, M. Yue, X. Sun, and X. Zhao, “Cooperative formation control strategy for multi-robot system based on APF algorithm and sliding mode estimator,” *Robot. Auton. Syst.*, vol. 193, p. 105075, 2025.
- [9] S. Macenski et al., “The Marathon 2: A navigation system,” IROS, 2020.
- [10] D. V. Lu et al., “Layered costmaps for context-sensitive navigation,” IROS, 2014.
- [11] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI*, CRC Press, 2018.
- [12] D. Faconti, “BehaviorTree.CPP” <https://www.behaviortree.dev>
- [13] M. Ahn et al., “Do as I can, not as I say: Grounding language in robotic affordances,” CoRL, 2022.
- [14] J. Liang et al., “Code as policies: Language model programs for embodied control,” ICRA, 2023.
- [15] I. Singh et al., “ProgPrompt: Generating situated robot task plans using large language models,” ICRA, 2023.
- [16] W. Huang et al., “Inner monologue: Embodied reasoning through planning with language models,” CoRL, 2022.
- [17] Y. Maruyama, S. Kato, and T. Azumi, “Exploring the performance of ROS2,” EMSOFT, 2016.
- [18] T. Kronauer, J. Pohlmann, M. Matthe, T. Smejkal, and G. Fetsch, “Latency analysis of ROS2 multi-node systems,” 2023.
- [19] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, 2022.

- [20] R. Vaughan, “Massively multi-robot simulation in Stage,” *Swarm Intell.*, vol. 2, pp. 189–208, 2008.
- [21] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” *IROS*, 2004.
- [22] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, “A survey of behavior trees in robotics and AI,” *Robot. Auton. Syst.*, vol. 154, 2022.
- [23] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” *NeurIPS*, 2023.
- [24] F. Zeng et al., “Large language models for robotics: A survey,” 2023.
- [25] A. Brohan et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv:2307.15818*, 2023.
- [26] D. Vizitei, A. Tuzov, D. Ryabov, “Hierarchical Multi-Agent Control by LLM: Simulation Environment, Per-Robot Navigation, and LLM Orchestration,” technical report, 2026.